nonce0 — complete design

Scanner, guard, and shielded destination as one system.

Status: design. Written 2026-09-07. Supersedes nothing; composes scanner-design.md, pqguard-spec.md, project-x-spec.md and integration-brief.md.

---

1. The system in one paragraph

Every Ethereum account is a keypair, and your address is a *hash* of the public key — so a quantum computer cannot attack it until the key itself is published. That happens the first time the account signs anything, on any chain. **nonce0** finds every account that controls your protocol and checks all of them across every chain, including the testnet where a developer burned the key two years ago. **PQGuard** adds a hash-based second signature to the authority path, so a broken ECDSA key is no longer sufficient to act. And **Project X** is where the funds go when you decide to move, because a fresh address is safe for exactly one transaction, and a mass migration would publish the old-to-new mapping for every wallet on the chain at once.

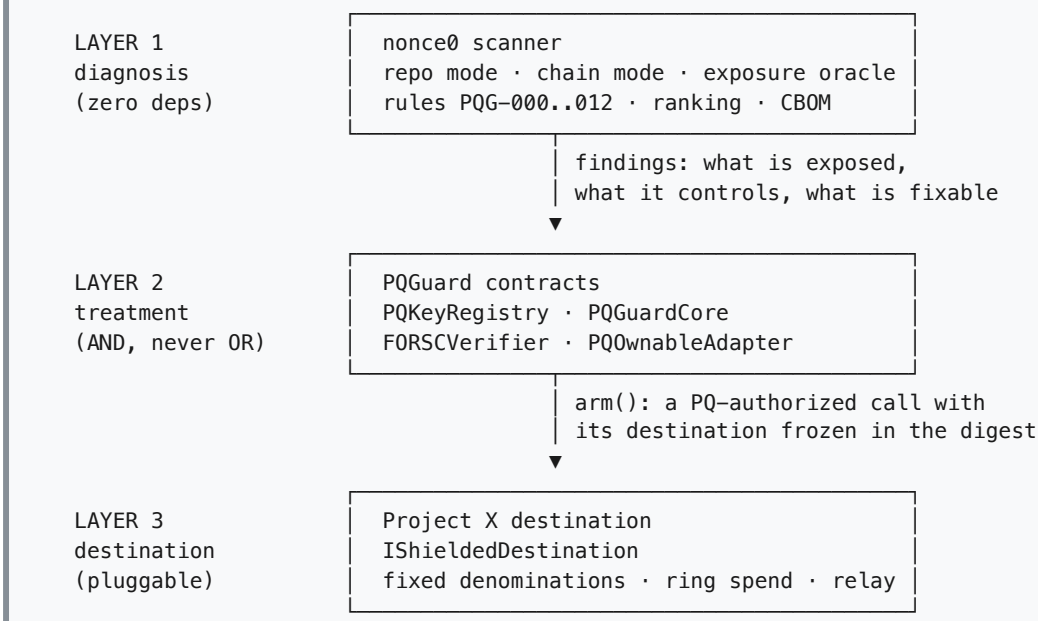
Three layers, three jobs:

Layer	Name	Job	Answers
1	**nonce0**	diagnosis	which of my keys are exposed, and what do they control?
2	**PQGuard**	treatment	how do I make the break insufficient, without redeploying?
3	**Project X**	destination	where do I move, and how do I move without publishing the map?

Removing any one leaves a real gap: layer 1 alone is a warning with no action, layer 2 alone protects an address you may not want to keep, layer 3 alone is a privacy pool nobody knows they need.

---

2. Layer map



The arrows only point down. Layer 1 knows nothing about layer 2; layer 2 knows nothing about layer 3 beyond an address and a calldata blob. That is what lets each layer be built, demoed and shipped without the ones below it existing.

---

3. Repo layout with the Project X layer

```
nonce0/                          the repo root IS the published npm package
├── bin/nonce0.js                arg parse, exit codes. ONLY file that may exit()
├── src/                         everything here RETURNS DATA AND THROWS
│   ├── scan.js                  the one orchestrator
│   ├── chains.js                8 chains, ordered RPC fallback, subgraph: null
│   ├── score.js                 risk = exposure x VaR x (1 - fixability)
│   ├── report.js                text reporter
│   ├── rules/{catalog,engine}.js
│   ├── repo/{discover,solidity,artifacts}.js
│   ├── chain/{rpc,disasm,slots,authority}.js
│   ├── exposure.js              eth_getTransactionCount across every chain
│   │
│   ├── pq/                      ── LAYER 2 client ──────────────────────────
│   │   ├── forsc.js             FORS+C keygen/sign. SHAKE256, node-native
│   │   ├── keystore.js          encrypted local key state, useCount tracking
│   │   └── digest.js            eth_call digestFor(); never hashes locally
│   │
│   └── migrate/                 ── LAYER 3 client ──────────────────────────
│       ├── plan.js              enumerate assets, classify into tiers
│       ├── note.js              note commitment (SHAKE256), denomination split
│       └── destinations.js      registry of IShieldedDestination addresses
│
├── contracts/                   PQGuard (layer 2) + the destination interface
│   ├── src/
│   │   ├── PQKeyRegistry.sol    key state, rotation, exhaustion, escape hatch
│   │   ├── PQGuardCore.sol      digest, replay, authorize, digestFor() view
│   │   ├── IPQVerifier.sol
│   │   ├── FORSCVerifier.sol    vendored w/ provenance header
│   │   ├── PQOwnableAdapter.sol becomes the owner and forwards
│   │   └── IShieldedDestination.sol   ── THE SEAM ──
│   └── test/
│       ├── MockVerifier.sol     lets layer 2 land before the crypto
│       ├── MockShieldedPool.sol lets layer 3 land before Project X
│       └── CRQCFork.t.sol       THE DEMO: same exploit before and after
│
├── projectx/                    ── LAYER 3 implementation (separate lane) ──
│   ├── contracts/               ring verifier, range proof, pool
│   └── relay/                   3-hop mesh, batching, randomized delay
│
├── backend/                     x402-gated scan API. ONLY node_modules
├── frontend/                    dashboard, no framework, no build
└── docs/
```

`projectx/` is a **directory, not a dependency**. Nothing in `src/` or `contracts/src/` imports from it. The connection is one address in `destinations.js` and one interface in `IShieldedDestination.sol`.

---

4. What the Project X layer adds

Layer 2 alone can protect an address. It cannot answer *should you keep that address*.

A fresh EOA is safe for exactly one transaction. Migrating an exposed key to a new EOA has a half-life of one spend: nonce 0 today, public key published the moment it signs. Any migration that ends at a transparent address is temporary by construction.

Mass migration is itself a deanonymization event. At Q-day everyone moves at once and every escape transaction is public: `0xABC → 0xDEF`, ten million times over a few weeks. The old-to-new mapping is a graph anyone builds in an afternoon, and every wallet's entire history follows it to the new address. The largest forced address migration in the history of the chain would also be the largest deanonymization event in it.

A shielded pool is the only destination that severs that link, because ten million edges into one pool address is not a mapping.

Stated precisely, because this is where projects overclaim: what is severed is **deposit → spend**, not **identity → deposit**. The arm transaction is public and the deposit is public. All anonymity lives in the ring at spend time. This is exactly why ring integrity (§9) is load-bearing rather than a nice-to-have.

4.1 Tiered destinations

Not everything can go into the pool, and the CLI must say so rather than implying uniform protection.

Asset	Destination	Protection
USDC and supported tokens	Project X shielded note	permanent, PQ-authorized, **unlinked**
Contract ownership	`PQOwnableAdapter` as owner	permanent; protocol address unchanged, owner linkable
NFTs, LP positions, vesting, long-tail tokens	fresh nonce-0 4337 account + PQ validator	permanent, **linkable**

> **EIP-7702 is not on this list, deliberately.** Delegation is authorized by an ECDSA-signed tuple `[chain_id, address, nonce, y_parity, r, s]`, processed *before* the execution portion of the transaction, and the EIP gives delegated code no mechanism to block a later re-delegation. A quantum adversary holding the key signs a fresh authorization at the next nonce and replaces your PQ validator. 7702 + PQ validator is **not permanent**, and anything claiming otherwise is wrong.

---

5. Where the cryptography happens

Node has **no keccak256** — only NIST SHA3, which is different padding. It does have SHA-256, SHA3, SHAKE128 and SHAKE256 with arbitrary output length. This single fact decides the client architecture.

Operation	Where	Why
FORS+C keygen	CLI, `src/pq/forsc.js`	hash-based; SHAKE256 is native, zero deps
FORS+C signing	CLI, `src/pq/forsc.js`	same
Note commitment	CLI, `src/migrate/note.js`	**define it over SHAKE256, never keccak**
PQGuard digest	on-chain, `eth_call digestFor(...)`	it is keccak, and it must match the contract byte-for-byte anyway
Ring signature verify	on-chain, layer 3	see §11 on hash choice and gas

```
// PQGuardCore — the view that keeps the CLI dependency-free.
function digestFor(
    address account, address target, uint256 value,
    bytes32 calldataHash, uint64 deadline
) external view returns (bytes32);
```

The CLI calls it, receives 32 bytes, signs those bytes with the PQ key locally, submits. It never needs a keccak implementation, and it can never disagree with the contract about what was signed.

> **Design rule: never define a commitment the CLI must compute over keccak256.** Choosing SHAKE256 for the note commitment costs the pool nothing and preserves the scanner's zero-dependency claim. Decide this before writing the pool, not after.

---

6. Initialization — `nonce0 init`

Bringing the PQ layer up. Runs once per protected account, and **while ECDSA still works**, which is the entire point: you are using a key that is currently trustworthy to commit to one that will still be trustworthy later.

```
$ nonce0 init --account 0xSafe --scheme fors-c

  1  generating FORS+C keypair                    SHAKE256, local, no network
  2  deriving commitment chain                    pk_0 -> H(pk_1) pre-committed
  3  writing keystore                             ~/.nonce0/0xSafe.keystore.json
  4  registering with PQKeyRegistry                tx 0x9a3c...  (ECDSA-authorized)

  pkCommitment    0x7f21...c40a
  nextCommitment  0x1b8e...93d1        pre-committed successor, hash-chained
  maxUses         64                   few-time budget, not a target
  rotationDeadline 2027-09-07

  BACK UP ~/.nonce0/0xSafe.keystore.json NOW.
  It holds the only copy of the commitment chain. Losing it means waiting out
  the 30-day escape hatch to disable the guard.
```

What happens on chain: one `registerKey(pkCommitment, nextCommitment, schemeId)` call, authorized by the existing ECDSA authority. Nothing is protected yet — registering a key does not enable a guard. That is a separate, deliberate second step:

```
$ nonce0 init --enable --account 0xSafe --adapter ownable
  transferOwnership(0xProtocol -> PQOwnableAdapter)   tx 0x44f1...
  guard live. protected selectors: upgradeTo, transferOwnership, grantRole, setImplementation
```

6.1 Key state, and why it is the hard part

The verifier is the easy part. Hash-based signatures are one-time or few-time, so a monotonic, replay-proof counter is a hard requirement:

```
struct KeyState {
    bytes32 pkCommitment;      // H(public key) of the active key
    bytes32 nextCommitment;    // pre-committed successor, hash-chained
    uint32  schemeId;          // resolves to an IPQVerifier
    uint32  useCount;          // monotonic, bound into every digest
    uint32  maxUses;           // hard cap, scheme-dependent
    uint64  rotationDeadline;
    uint64  disableAfter;      // 0 = enabled; else escape-hatch unlock time
}
```

`useCount` is bound into the signed digest, so **a signature is valid at exactly one index**. The CLI reads `useCount` from the chain before every signature; the registry is the single source of truth, never the local keystore.

Rotation is authenticated by the current PQ key, never by ECDSA. Otherwise the quantum adversary rotates your key for you and the whole layer is theatre.

The escape hatch is `requestDisable()`, callable by the existing ECDSA authority, starting a 30-day timelock. A quantum adversary holding your ECDSA key can start it too — which is precisely why the delay is measured in weeks and emits an event on every block explorer. Every block of that window is a public alarm.

---

7. Migration — `nonce0 migrate`

Three subcommands, and they are deliberately three separate transactions on three separate days if you want them to be.

7.1 `migrate --plan` — what would move, and where

Read-only. No keys, no transactions.

```
$ nonce0 migrate --plan 0xTreasury

  SHIELDED  (permanent, unlinked)
    USDC          1,240,000.00   →  10 notes x 100,000  +  4 x 10,000  +  remainder 0
    USDT             85,000.00   →   8 notes x 10,000   +  remainder 5,000  ← see below

  ADAPTER   (permanent, protocol address unchanged, owner linkable)
    0xProtocol   upgrade authority  →  PQOwnableAdapter

  FRESH     (permanent, linkable)
    3 NFTs, 1 Uniswap v3 position, 1 vesting contract
    → these cannot enter the pool. A transparent swap first is itself a linkable action.

  REMAINDER 5,000 USDT does not fit a fixed denomination. Options:
    (a) leave it, (b) round down and abandon it, (c) swap to a denomination first.
    A non-standard note amount links your deposit to your withdrawal REGARDLESS
    of ring size. This is not optional advice.
```

Fixed denominations are a hard requirement, not a nicety. Deposit 13.47 and withdraw 13.47 and the amounts link the two regardless of the ring. The plan output exists to make that visible before anyone commits.

7.2 `migrate --arm` — commit the destination, move nothing

The critical step. Run it **today**, while ECDSA is still trustworthy.

```
$ nonce0 migrate --arm --account 0xTreasury --to shielded

  1  reading useCount from PQKeyRegistry                 useCount = 0
  2  deriving note commitments                           SHAKE256, local
  3  building calldata   deposit(0x8f2a…, 100000e6)      x10
  4  eth_call PQGuardCore.digestFor(...)                 digest 0x3d91…
  5  signing digest with FORS+C                          local, useCount 0
  6  submitting arm(digest, signature)                   tx 0xbe07…  ECDSA-authorized

  ARMED. Nothing has moved.
    destination   0xPool  (pinned)
    executable    any time before 2036-09-07
    redirectable  NO — target and calldata hash are inside the signed digest
```

Why this is safe to leave sitting on chain:

- **The destination cannot be redirected.** `target` and `keccak256(callData)` are bound into the digest. A different pool or a different note is a different digest, and the PQ signature does not verify against it.
- **Front-running gains nothing.** The armed call is public, but executing it requires a signature only the key holder has, and the call does exactly one thing.
- **The ECDSA key is used only here, while it still works.** After arming, no ECDSA signature appears anywhere in the path.

> Deliberate deviation from `pqguard-spec.md` §6.1, which defines `arm()` as short-lived (single block or short deadline). Migration wants arm-now / execute-at-Q-day. Long deadlines are safe *because* the digest pins the target, but the deadline must still be bounded — a decade, not `type(uint64).max`.

7.3 `migrate --execute` — move, PQ-authorized

```
$ nonce0 migrate --execute --account 0xTreasury

  1  reading armed digest                        0x3d91…  still valid
  2  reading useCount                            useCount = 0  (matches arm)
  3  PQOwnableAdapter.execute(target, value, data, deadline, envelope)
       → PQGuardCore.authorize()   FORS+C verified, useCount -> 1
       → IShieldedDestination.deposit(0x8f2a…, 100000e6)
                                                 tx 0x77c2…

  MIGRATED. 10 notes deposited. Secrets are in ~/.nonce0/0xTreasury.notes.json
  Without that file the notes are unspendable. Back it up separately from the keystore.
```

7.4 The whole flow, end to end

```
nonce0 scan 0xProtocol          "your admin key has nonce 47 on Sepolia — exposed"
      │
      ▼
nonce0 init                     PQ keypair, commitment registered   [ECDSA still works]
      │
      ▼
nonce0 init --enable            PQOwnableAdapter becomes the owner  [ECDSA still works]
      │
      ▼
nonce0 migrate --plan           what moves, to which tier, what is stranded
      │
      ▼
nonce0 migrate --arm            destination pinned in the digest    [ECDSA still works]
      │
      │      ····· Q-day. ECDSA is now forgeable. ·····
      │
      ▼
nonce0 migrate --execute        PQ-signed. No ECDSA anywhere in this path.
```

The property that matters: **every step requiring ECDSA happens before the break, and every step after the break requires only the hash-based key.** An adversary who breaks ECDSA on Q-day arrives to find the destination already committed and unforgeable.

7.5 What an adversary can and cannot do after the break

Adversary action	Outcome
Forge an ECDSA signature from the admin key	Denied — `PQGuardCore` requires the PQ signature too (AND, never OR)
Redirect the armed migration to their own address	Impossible — different target is a different digest
Front-run `--execute`	Pointless — the call deposits to the pinned note either way
Call `requestDisable()` to strip the guard	Starts a **30-day public timelock**, alarming on every explorer
Replay the arm signature at a different `useCount`	Denied — `useCount` is bound into the digest
Trace the funds after deposit	Only to the pool. Spend is ring-authorized

---

8. The seam

One interface, three valid implementations: a mock pool, an existing pool, Project X.

```
interface IShieldedDestination {
    /// @param noteCommitment  opaque to the guard. Defined over SHAKE256, not keccak.
    /// @param denomination    MUST be one of denominations()
    function deposit(bytes32 noteCommitment, uint256 denomination) external payable;

    /// Fixed denominations. A non-standard amount links deposit to withdrawal
    /// regardless of ring size, so the guard refuses to arm one.
    function denominations() external view returns (uint256[] memory);
}
```

The digest that pins it, already specified in `pqguard-spec.md` §4:

```
payload = abi.encode(target, selector, keccak256(callData), value, deadline)
digest  = keccak256(abi.encode(
              PQ_DOMAIN,          // distinguishes from EIP-712 and everything else
              block.chainid,      // no cross-chain replay of governance actions
              address(core),      // PQGuardCore, pinned
              account,
              schemeId,           // no downgrade replay across verifiers
              useCount,           // monotonic, one signature per index
              keccak256(payload)  // target + calldata: the destination
          ))
```

Every field is load-bearing. Drop `schemeId` and a downgrade replay across verifiers becomes possible; drop `useCount` and the one-time property breaks; drop `chainid` and a governance action replays on another chain.

Layer 2 never learns what a note is. It sees an address and a calldata hash. That is the whole coupling, and it is why swapping a mock pool for Project X changes one constant in `src/migrate/destinations.js`.

---

9. Ring integrity — what the scanner tells the pool

The one capability neither project has alone, and it needs no lattice cryptography.

> A ring of 8 whose members' keys are already exposed does not have an anonymity set of 8. A quantum adversary derives those private keys, rules those members out, and the ring collapses to the members still hidden.

```
$ nonce0 ring 0xPool

  ring members reported      8
  keys already exposed       6      4 mainnet, 2 testnet-only
  effective anonymity        2      ← what a quantum adversary actually faces

  exposed members
    0x3f9a…  nonce 214 on ethereum      first exposed 2019-04-11
    0x77c1…  nonce   1 on sepolia       first exposed 2024-08-02   ← testnet only
    …
```

Computed entirely from the existing exposure oracle plus authority traversal: the same engine as the v4 hook ecosystem scan, pointed at a new target set. It works against Railgun, Tornado-shaped pools and Project X identically.

State it as an upper bound, always. It measures anonymity lost to *key exposure*. It does not measure anonymity lost to timing correlation, amount correlation, or common funding. Effective anonymity is *at most* this number and possibly lower.

This is also the honest answer to "why is a scanner in a privacy project": a pool cannot measure its own anonymity set, because doing so requires cross-chain exposure data that only the scanner has.

---

10. Contracts

Contract	Layer	Responsibility
`PQKeyRegistry`	2	Key state, hash-chained rotation, monotonic `useCount`, 30-day timelocked disable. Rotation authenticated by the current PQ key, never ECDSA.
`PQGuardCore`	2	Domain-separated digest, `digestFor()` view, replay, verifier dispatch, counter increment. **AND composition: it can only ever deny, never grant.**
`IPQVerifier`	2	Stateless `verify(digest, sig, pkCommitment) → (ok, revealedNext)`. All state lives in the registry.
`FORSCVerifier`	2	Default scheme. ~2448-byte signatures, ~35k verification gas. Vendored from an audited reference with a provenance header; do not write the primitive.
`PQOwnableAdapter`	2	Becomes the owner and forwards. One-transaction migration. Also the `arm()` / `execute()` host for §7.
`IShieldedDestination`	3	The seam.
`MockShieldedPool`	test	Fixed denominations, no ring. Lets layer 3 be exercised before Project X exists.
`CRQCFork.t.sol`	test	**The demo.** An oracle returning the private key for any exposed public key; identical exploit before and after install.

The strongest structural property: because PQGuard only ever *adds* a requirement, a bug in it cannot authorize anything. The worst case is a liveness failure with a documented, timelocked recovery — not a loss of funds. Say this plainly; it is what makes adopting an unproven module rational.

Overhead is ~89,000 gas per protected call (39k calldata, 35k verification, 15k state). Irrelevant for a governance action executed a few times a year, prohibitive per swap. That is a scoping argument, not a limitation to apologise for: the assets a quantum adversary takes are taken through the authority path, not the trading path.

---

11. Layer 3 feasibility — the one open engineering risk

Stated plainly because it is the only part of this design that is not routine.

ETHDILITHIUM / ETHFALCON, **single** signer	1.9M – 8.8M gas
Ring of 8, if verification is linear in ring size	**15M – 70M gas**
Ethereum mainnet block gas limit	**60M** (measured, block 25,925,286)
Calldata, 45.6KB at ring 8, post-EIP-7623 floor	~1.8M gas (minor by comparison)

It straddles the limit. Three levers, in order of impact:

1. **The hash function.** ChipmunkRing's verification is iterative **SHAKE256**, which is *not* an EVM precompile — verifying it on-chain means implementing keccak-f[1600] in Solidity. Compare: `keccak256` is an **opcode** (30 gas + 6/word), `SHA-256` is a **precompile** at `0x02` (60 gas + 12/word). Re-instantiating the construction over either plausibly moves this by an order of magnitude. *Honest caveat:* that ships a **variant** of the published scheme, not the scheme. For a construction proved in the random-oracle model this is usually sound, but say so rather than implying a port.
2. **Verify on Arc, not L1.** Settlement is already there. Nothing requires ring verification to happen on mainnet, and this turns a borderline number into a comfortable one. **This is the recommended path.**
3. **Ring 4 instead of 8.** Halves signature size and roughly halves the work. The spec already calls ring 8 "a target, not a guarantee."

The experiment worth running before committing: measure one ring-4 verification on the cheap hash. Above ~20M gas even then, layer 3 is a research project rather than a build, and the mock pool becomes the shipped destination.

---

12. CLI surface

```
nonce0 scan <path>                  repo mode: source, pre-deploy
nonce0 scan 0xProtocol              chain mode: bytecode + authority graph
nonce0 ring 0xPool                  effective vs reported anonymity
nonce0 init                         PQ keypair + registry commitment
nonce0 init --enable                install the adapter as owner
nonce0 migrate --plan               what moves, to which tier
nonce0 migrate --arm --to shielded  pin the destination, move nothing
nonce0 migrate --execute            PQ-signed migration
nonce0 verify                       is the guard live and armed?
nonce0 mcp                          stdio MCP server for assistants

  --json              raw findings, for tooling and the MCP server
  --fail-on <sev>     exit 1 at or above this severity (default: critical)
  --include-deps      also scan vendored dependencies

exit 0 clean · 1 findings at or above --fail-on · 2 tool error
```

Exit 2 must never be reachable by a scan that merely found nothing: **a network failure must never read as a clean scan.**

---

13. Threat model

The system hides: which ring member spent a note; the note amount (fixed denominations, so amounts carry no entropy); the sender's network origin at deposit and spend, via the relay mesh; the input evaluated by the confidential policy check.

The system protects: the authority path, against an adversary who can derive any secp256k1 private key from any published public key.

The system does NOT claim to prevent:

- a global passive network adversary;
- simultaneous compromise of every relay hop (network-origin privacy only — never part of the payment-validity trust path);
- weak OPSEC: timing and amount patterns outside the protocol;
- **retroactive decryption.** Every ECIES note and ECDH stealth address already on chain is permanently exposed. No module fixes this;
- **immutable verifier soundness.** If a deployed Groth16 verifier is your mint authority, this cannot make it sound. It can only cap the drain rate.

Trust assumptions: none for payment validity — the ring signature is verified by the contract. None for authority — PQGuard can only deny. The relay mesh is trusted *only* for network-origin privacy.

The adversary model, said out loud: a quantum adversary that derives any secp256k1 private key from any on-chain-exposed public key, in the time it takes to run a transaction, with no preimage or collision advantage on 256-bit hashes beyond Grover.

---

14. Honesty guardrails

Every one of these is a place a competent judge probes, and the credible parts of the project go down with an overclaim.

Do not say	Say
"Quantum-secure"	"The **authority path** is post-quantum. The wallet layer is still ECDSA — an Ethereum-wide gap we do not fix."
"Anonymity set of 8"	"Ring size 8, a PoC number. Our own tool reports the **effective** anonymity, which is lower."
"TEE-attested"	"Attestation is **simulated** in this build; the hardware path is unchanged but unverified here."
"We broke ECDSA"	"Our adversary is an **oracle that already holds the private key**. We show what happens after a break, not a break."
"Migration is anonymous"	"It severs **deposit** → **spend**. The deposit is public. Anonymity lives in the ring at spend time."
"Effective anonymity is 2"	"**At most** 2 — key exposure only, not timing or amount correlation."
"We protect your funds"	"PQGuard can only **deny**, never grant. Worst case is stuck admin calls and the recovery timer."
"We ported ChipmunkRing"	"A **variant** instantiated over a different hash, for EVM cost. Not the published scheme."

15. Failure modes

Failure	Consequence	Mitigation
PQ key lost	Protected calls blocked	30-day timelocked disable, loud events, offline backup of the commitment chain
Note secrets lost	Notes unspendable — **funds gone**	`~/.nonce0/*.notes.json` backed up separately from the keystore. Say this at `--execute` time, loudly
Key state desync	Signature rejected at the wrong index	Registry is the single source of truth; CLI reads `useCount` before every signature
Accidental key reuse	Partial key material exposure	FORS+C degrades gracefully; WOTS+C does not, hence the default
Authority traversal returns empty	Clean bill of health that is false	Explicit `PQG-000 unresolved authority` finding on any probe not understood
RPC failure during exposure check	"not exposed" that is actually unknown	Three-valued result: `exposed | not-exposed | unknown`, and unknown is loud
Lattice verifier infeasible	No shielded destination	Mock pool is the shipped destination; interface unchanged
Guard itself is buggy	Protocol frozen	AND composition means it cannot grant. Escape hatch recovers

16. Open questions

1. **Which chain verifies the ring signature?** Recommended: Arc, where settlement already happens. Deciding this changes the gas budget by an order of magnitude.
2. **Which hash instantiates the construction?** SHAKE256 is the paper; keccak256 or SHA-256 are the EVM. This is the single highest-leverage decision in layer 3.
3. **Who owns layer 3?** If the same people own layers 1–2, layer 3 is the mock pool and the pitch is "we defined the interface and built against it" — still coherent, but a different sentence on camera.